

Análisis algorítmico de ALNS y ALNS con Q-learning para el VRPTW

Estructura de datos, métodos de selección, operadores y optimizaciones

Documento técnico de apoyo a la tesis

9 de octubre de 2026

Resumen

Este documento describe, a nivel de pseudocódigo y con su justificación teórica, la implementación de los dos métodos comparados en la tesis para el problema de ruteo de vehículos con ventanas de tiempo (VRPTW): un ALNS con selección adaptativa por ruleta (**ALNS_Classic**) y un ALNS cuya selección de operadores se aprende con Q-learning (**ALNS_QLearning**). Se detallan el bucle de búsqueda común en dos fases (minimización de rutas y minimización de distancia), los dos mecanismos de selección y aprendizaje, los seis operadores de destrucción, los cinco de reparación y las optimizaciones algorítmicas que determinan el costo por iteración. Todos los valores numéricos corresponden a las constantes y parámetros por defecto del código fuente.

Índice

1. Alcance y organización del código	3
2. Modelo de datos y evaluación de factibilidad	3
2.1. Instancia	3
2.2. Ruta	4
2.3. Evaluación de una inserción en tiempo constante	5
2.4. Solución y función de costo	5
2.5. Solución inicial	6
3. Bucle de búsqueda común	6
3.1. Las dos fases de la búsqueda	8
3.2. Ganancias relativas	9
3.3. Criterio de aceptación	9
3.4. Esquema de temperatura	10
3.5. Catálogo de operadores y grado de destrucción	10
4. ALNS clásico: selección por ruleta adaptativa	11
4.1. Selección	12
4.2. Asignación de crédito y actualización	12
4.3. Propiedades relevantes	12
5. ALNS con Q-learning	13
5.1. Selección de la acción	14
5.2. Función de recompensa	15
5.3. Actualización de la tabla	16
5.4. Reacción al cambio de fase	16
5.5. Propiedades relevantes	17

6. Operadores de destrucción	17
6.1. Extracción aleatoria	18
6.2. Extracción de rutas	18
6.3. Extracción de los peores	18
6.4. Extracción de Shaw	19
6.5. Extracción de grupos	20
6.6. Extracción por ventana de tiempo	21
7. Operadores de reparación	22
7.1. Formulación directa y su costo	22
7.2. Inserción con caché por ruta	22
7.3. Ruido en los costos de inserción	23
7.4. Agregadores y actualización perezosa	24
7.5. Resumen de las optimizaciones	25
8. Síntesis: características que determinan el comportamiento	26
8.1. Características comunes	26
8.2. Diferencias entre los dos métodos	26

Índice de algoritmos

1. RECALCULARRUTA (<code>Route::recalculate</code>)	4
2. EVALUARINSERCIÓN (<code>evalInsertion</code>)	5
3. SOLUCIÓNINICIAL (<code>Solution::generateInitialSolution</code>)	6
4. Bucle ALNS común en dos fases (<code>ALNS::solve</code>)	7
5. INICIARETAPA (función local <code>startStage</code> de <code>ALNS::solve</code>)	7
6. APLICAROPERADORES (<code>applyOperators</code>)	11
7. Selección y aprendizaje del ALNS clásico (<code>ALNS_Classic</code>)	12
8. Selección ε -voraz (<code>ALNS_QLearning::selectOperators</code>)	14
9. Aprendizaje por Q-learning (<code>ALNS_QLearning::learn</code>)	16
10. Inicio de la fase 2 en el Q-learning (<code>ALNS_QLearning::onDistancePhase</code>)	16
11. EXTRACCIÓNALEATORIA (<code>randomRemoval</code>)	18
12. EXTRACCIÓNDERUTAS (<code>routeRemoval</code>)	18
13. EXTRACCIÓNDEPEORES (<code>worstRemoval</code>)	19
14. EXTRACCIÓNDESHAW (<code>shawRemoval</code>)	20
15. EXTRACCIÓNDEGRUPOS (<code>clusterRemoval</code>)	21
16. EXTRACCIÓNPORVENTANA (<code>timeWindowRemoval</code>)	21
17. INSERCIÓNCONCACHÉ $\langle G \rangle$ (<code>cachedInsertion<Agg></code>)	23
18. ENTRADA (<code>computeEntry</code>)	23

1. Alcance y organización del código

El solver está escrito en C++ y separa con claridad tres responsabilidades: la representación del problema, los operadores de vecindario y el control de la búsqueda. La Tabla 1 resume qué contiene cada módulo y en qué sección de este documento se analiza.

Tabla 1: Módulos del solver y sección donde se documentan.

Archivo	Contenido	Sección
VRPTW Environment/instance.*	Instancia, cliente, ruta; matrices precalculadas; recálculo de una ruta	2
VRPTW Environment/solution.*	Solución, función de costo, solución inicial	2
Utils/params.h	Parámetros ajustables del Q-learning (SolverParams)	5
ALNS/alns.*	Bucle común en dos fases (rutas y distancia): recocido simulado, aceptación, reinicio por estancamiento	3
ALNS/alns_classic.*	Selección por ruleta con pesos adaptativos	4
ALNS/alns_qlearning.*	Selección ε -voraz sobre una tabla Q	5
Operators/operators.*	Catálogo de operadores y grado de destrucción	3
Operators/destroy_ops.cpp	Seis operadores de destrucción	6
Operators/repair_ops.cpp	Evaluación de inserción y cinco operadores de reparación	7

La clase abstracta **ALNS** implementa el bucle de búsqueda completo y delega únicamente dos decisiones en sus subclases, mediante los métodos virtuales **selectOperators** y **learn**. Un tercer método virtual, **onDistancePhase**, es solo un aviso de que comienza la segunda fase de la búsqueda (Sección 3.1); por defecto no hace nada. En consecuencia, **ALNS_Classic** y **ALNS_QLearning** comparten *exactamente* la misma solución inicial, los mismos operadores, el mismo criterio de aceptación, el mismo esquema de temperatura, la misma división en fases y el mismo generador de números aleatorios (`std::mt19937`, sembrado con la misma semilla para ambos algoritmos en cada corrida). Lo único que cambia entre los dos métodos es *qué par de operadores se elige en cada iteración y cómo se actualiza esa preferencia*. Esta propiedad de diseño es la que permite atribuir las diferencias de resultados al mecanismo de selección y no a otro componente.

Notación. Se usa n para el número de clientes, m para el número de rutas de una solución, $\bar{L} = n/m$ para la longitud media de una ruta, q para el número de clientes que se extraen en una iteración (grado de destrucción) y $T_{\max}$ para el presupuesto de iteraciones que recibe el solver. $T_{\max}$ es el número exacto de iteraciones de la fase de minimización de distancia; la fase previa de minimización de rutas consume, como máximo, otras $T_{\max}$, de modo que una corrida ejecuta entre $T_{\max}$ y $2T_{\max}$ iteraciones. En los experimentos de la tesis $T_{\max} = 25\,000$ y cada instancia se resuelve con 10 semillas por algoritmo.

2. Modelo de datos y evaluación de factibilidad

2.1. Instancia

Una instancia (**Instance**) se lee en el formato de Solomon [7], que también usan las instancias de Gehring y Homberger [1]. Contiene el depósito (nodo 0) y n clientes ($N = n + 1$ nodos). Cada nodo i tiene coordenadas (x_i, y_i) , demanda μ_i , ventana de tiempo $[a_i, b_i]$ (**ready_time**, **due_date**) y tiempo de servicio s_i . Todos los vehículos tienen capacidad Q . El archivo declara además un tamaño de flota, que el solver lee pero *no impone* como restricción: el número de vehículos es un objetivo, no un límite. La flota con la que trabaja cada etapa de la búsqueda la fija el propio algoritmo (Sección 3.1).

Al construir la instancia se precálculan dos matrices $N \times N$ que se consultan en tiempo constante durante toda la búsqueda:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}, \quad (1)$$

$$\text{alc}(i, j) = [i \neq j \wedge a_i + s_i + d_{ij} \leq b_j]. \quad (2)$$

La distancia (1) es euclidiana en doble precisión, sin redondeo ni truncamiento, y coincide con el tiempo de viaje. La matriz de alcanzabilidad (2) indica si es posible visitar j inmediatamente después de i en el mejor caso (saliendo de i lo antes posible); si $\text{alc}(i, j)$ es falso, el arco (i, j) no puede aparecer en ninguna ruta factible. Es una condición necesaria, no suficiente, y se usa como filtro previo barato. Durante el mismo precálculo se guarda la mayor distancia entre dos nodos, $d_{\text{máx}} = \max_{i,j} d_{ij}$ (**max_dist**), que fija la amplitud del ruido de los operadores de reparación (Sección 7.3).

La instancia guarda además una tercera estructura, **shaw_order**, que se construye de forma perezosa la primera vez que se necesita y se describe en la Sección 6.4.

2.2. Ruta

Una ruta (**Route**) es una secuencia $\pi = (\pi_0, \pi_1, \dots, \pi_{L-1})$ con $\pi_0 = \pi_{L-1} = 0$. Una ruta con $L = 2$ está vacía y no cuenta como vehículo usado. Junto a la secuencia se almacenan su carga, su distancia y tres vectores de longitud L :

- A_k : instante de llegada a π_k (**arrival_times**);
- $W_k = \max(0, a_{\pi_k} - A_k)$: espera en π_k hasta que abre su ventana (**wait_times**);
- F_k : holgura temporal hacia adelante (**time_slacks**), es decir, cuánto puede retrasarse el inicio del servicio en π_k sin violar la ventana de π_k ni la de ningún nodo posterior de la ruta.

El inicio del servicio en π_k es $\max(A_k, a_{\pi_k})$. La holgura sigue la recurrencia clásica de Savelsbergh [5]:

$$F_{L-1} = b_0 - \max(A_{L-1}, a_0), \quad F_k = \min(b_{\pi_k} - \max(A_k, a_{\pi_k}), F_{k+1} + W_{k+1}). \quad (3)$$

El segundo término expresa que un retraso en π_k se transmite al nodo siguiente, donde primero consume la espera W_{k+1} (tiempo que de todos modos se iba a perder) y solo el excedente consume su holgura F_{k+1} .

El Algoritmo 1 recalcula todos estos valores con una pasada hacia adelante y otra hacia atrás, en tiempo $\mathcal{O}(L)$. Se invoca cada vez que una ruta cambia.

Algoritmo 1: RECALCULARRUTA (**Route::recalculate**)

Entrada : ruta π de longitud L , instancia

Salida : carga, distancia y vectores A , W , F de la ruta

```

1  $carga \leftarrow 0$ ;  $dist \leftarrow 0$ ;  $A_0 \leftarrow 0$ ;  $W_0 \leftarrow 0$ 
2 para  $k \leftarrow 0$  hasta  $L - 2$  hacer
3    $u \leftarrow \pi_k$ ;  $v \leftarrow \pi_{k+1}$ 
4    $dist \leftarrow dist + d_{uv}$ 
5   si  $u \neq 0$  entonces  $carga \leftarrow carga + \mu_u$ 
6    $A_{k+1} \leftarrow \max(A_k, a_u) + s_u + d_{uv}$   $\triangleright$  inicio de servicio en  $u$  + servicio + viaje
7    $W_{k+1} \leftarrow \max(0, a_v - A_{k+1})$ 
8  $F_{L-1} \leftarrow b_0 - \max(A_{L-1}, a_0)$ 
9 para  $k \leftarrow L - 2$  hasta  $0$  hacer
10   $F_k \leftarrow \min(b_{\pi_k} - \max(A_k, a_{\pi_k}), F_{k+1} + W_{k+1})$ 
```

2.3. Evaluación de una inserción en tiempo constante

La operación más frecuente de todo el solver es preguntar si un cliente u puede insertarse entre π_k y π_{k+1} y cuánto aumenta la distancia. Con los vectores A , W y F la respuesta se obtiene en $\mathcal{O}(1)$, sin recorrer el resto de la ruta (Algoritmo 2).

Algoritmo 2: EVALUARINSERCIÓN (evalInsertion)

Entrada : cliente u , ruta π con A, W, F vigentes, posición k

Salida : (*factible*, Δ)

```

1  $i \leftarrow \pi_k; \quad j \leftarrow \pi_{k+1}$ 
2 si  $\neg \text{alc}(i, u)$  o  $\neg \text{alc}(u, j)$  entonces devolver (falso, -)
3  $A_u \leftarrow \max(A_k, a_i) + s_i + d_{iu}$   $\triangleright$  llegada a  $u$ 
4 si  $A_u > b_u$  entonces devolver (falso, -)
5  $A'_j \leftarrow \max(A_u, a_u) + s_u + d_{uj}$   $\triangleright$  nueva llegada a  $j$ 
6  $\text{retraso} \leftarrow \max(0, A'_j - A_{k+1})$ 
7 si  $\text{retraso} > W_{k+1} + F_{k+1}$  entonces devolver (falso, -)
8  $\Delta \leftarrow d_{iu} + d_{uj} - d_{ij}$ 
9 devolver (verdadero,  $\Delta$ )
```

La corrección de la prueba se sigue de la definición de F : el retraso en la llegada a j es absorbido primero por la espera W_{k+1} , y el inicio del servicio en j se desplaza en $\max(0, \text{retraso} - W_{k+1})$, que por (3) es admisible para j y para todos los nodos posteriores si y solo si no supera F_{k+1} . La restricción de capacidad no depende de la posición, por lo que se verifica una sola vez por ruta ($\text{carga} + \mu_u \leq Q$) antes de recorrer sus posiciones.

Evaluar todas las posiciones de una ruta cuesta entonces $\mathcal{O}(L)$ en lugar de $\mathcal{O}(L^2)$, y evaluar un cliente contra toda la solución cuesta $\mathcal{O}(n + m)$. El precio es que cada modificación de una ruta exige un recálculo $\mathcal{O}(L)$ (Algoritmo 1), que es despreciable frente al número de evaluaciones que habilita.

2.4. Solución y función de costo

Una solución (Solution) x contiene el vector de rutas $\mathcal{R}(x)$, la lista $\mathcal{U}(x)$ de clientes sin asignar (unassigned) y dos métricas agregadas que mantiene updateMetrics: el número de vehículos $K(x)$, que cuenta solo las rutas no vacías, y la distancia total $D(x)$ de esas rutas. Una solución es *completa* si $\mathcal{U}(x) = \emptyset$.

Todas las rutas son factibles en todo momento: los operadores de destrucción solo quitan clientes (lo que nunca rompe la factibilidad cuando se cumple la desigualdad triangular) y los de reparación solo ejecutan inserciones aceptadas por el Algoritmo 2. La búsqueda nunca visita soluciones con violaciones de capacidad o de ventanas; la única forma de infactibilidad posible es que queden clientes sin asignar, y la función de costo la penaliza de forma explícita.

El objetivo jerárquico habitual del VRPTW (primero minimizar vehículos, después distancia) se escalariza con un peso fijo por vehículo, al que se antepone una penalización por cada cliente sin asignar:

$$f(x) = 10^6 \cdot |\mathcal{U}(x)| + 10^4 \cdot K(x) + D(x). \quad (4)$$

Mientras los pesos dominen a los términos siguientes (en particular, mientras $D(x) < 10^4$ por cada vehículo de diferencia), comparar f equivale a la comparación lexicográfica $(|\mathcal{U}|, K, D)$: primero colocar a todos los clientes, después usar menos vehículos y por último recorrer menos distancia. Gracias al primer término, f está definida también para soluciones incompletas, que por tanto participan en la búsqueda como cualquier otra; esto es lo que permite la fase de minimización de rutas de la Sección 3.1. La función f es la que usan el criterio de aceptación y la detección de mejoras; las ganancias que alimentan el aprendizaje se miden con el mismo orden lexicográfico, pero en términos relativos (Sección 3.2).

2.5. Solución inicial

La solución inicial se construye con una heurística secuencial del vecino más cercano (Algoritmo 3): cada ruta parte del depósito y agrega repetidamente el cliente no visitado más cercano al último nodo que sea alcanzable, respete la capacidad, llegue dentro de su ventana y permita regresar al depósito antes de b_0 . Cuando ningún cliente cumple estas condiciones la ruta se cierra y se abre otra. Es determinista, por lo que ambos algoritmos y todas las semillas parten de la misma solución; su calidad es deliberadamente modesta y toda la mejora posterior corresponde al ALNS.

Algoritmo 3: SOLUCIÓNINICIAL (Solution::generateInitialSolution)

```

Entrada : instancia
Salida   : solución  $x$  con rutas factibles
1 mientras queden clientes sin visitar hacer
2    $\pi \leftarrow (0, 0); \quad c \leftarrow 0; \quad t \leftarrow 0; \quad carga \leftarrow 0$ 
3   mientras verdadero hacer
4      $\mathcal{C} \leftarrow \{ i \text{ no visitado} : \text{alc}(c, i), \text{ carga} + \mu_i \leq Q, t_i \leq b_i, t_i + s_i + d_{i0} \leq b_0 \}$ 
5     donde  $t_i = \max(t + s_c + d_{ci}, a_i)$ 
6     si  $\mathcal{C} \neq \emptyset$  entonces
7        $i^* \leftarrow \arg \min_{i \in \mathcal{C}} d_{ci}$ 
8       insertar  $i^*$  al final de  $\pi$ ; marcarlo visitado
9        $carga \leftarrow carga + \mu_{i^*}; \quad t \leftarrow t_{i^*}; \quad c \leftarrow i^*$ 
10    si no
11      si  $c = 0$  entonces enviar a  $\mathcal{U}$  todos los clientes no visitados  $\triangleright$  no caben ni solos
12      salir del bucle
13  RECALCULARRUTA( $\pi$ ); agregar  $\pi$  a  $\mathcal{R}(x)$ 
14 actualizar  $K(x)$  y  $D(x)$ 
```

3. Bucle de búsqueda común

La búsqueda adaptativa en vecindarios grandes (ALNS) [4, 3] mejora una solución destruyéndola parcialmente y reparándola en cada iteración. El Algoritmo 4 reproduce el método ALNS::solve, común a los dos métodos de la tesis. Las llamadas SELECCIONAR y APRENDER son los dos únicos puntos de variación (Secciones 4 y 5).

La búsqueda se organiza en *dos fases*, siguiendo el esquema de Ropke y Pisinger [4]: la fase 1 minimiza el número de rutas y la fase 2 minimiza la distancia con la flota obtenida. Ambas ejecutan exactamente el mismo cuerpo de iteración; lo que cambia es la solución de la que se parte y las condiciones de corte (Sección 3.1). En el pseudocódigo, $\bar{x}$ denota la última solución completa conocida y T_{fin} la última iteración, que se fija al comenzar la fase 2.

Algoritmo 4: Bucle ALNS común en dos fases (ALNS::solve)

Entrada : solución inicial x_0 , presupuesto de iteraciones $T_{\text{máx}}$
Salida : mejor solución encontrada x^*

```
1  $\bar{x} \leftarrow x_0$ ;  $\text{fase1} \leftarrow \text{verdadero}$ ;  $T_{\text{fin}} \leftarrow 2T_{\text{máx}}$ 
2  $T_0 \leftarrow -w D(x_0)/\ln 0.5$ ;  $K_{\text{mín}} \leftarrow \lceil \sum_i \mu_i / Q \rceil$ 
3 INICIARETAPA(0) ▷ Algoritmo 5: fija  $x$ ,  $x^*$ ,  $T$  y los contadores
4  $t \leftarrow 1$ 
5 mientras  $t \leq T_{\text{fin}}$  hacer
6    $x' \leftarrow x$  ▷ copia de trabajo
7    $(j, k) \leftarrow \text{SELECCIONAR}(t)$  ▷ destrucción  $j$ , reparación  $k$ 
8    $\text{APLICAROPERADORES}(x', j, k)$  ▷ Algoritmo 6
9    $b_{\text{ant}} \leftarrow |\mathcal{U}(x^*)|$ 
10   $o.g^{\text{mej}} \leftarrow G(x^*, x')$ ;  $o.g^{\text{act}} \leftarrow G(x, x')$  ▷ ganancias relativas, ec. (6)
11   $o.\text{nuevoMejor} \leftarrow f(x') < f(x^*)$ 
12   $o.\text{mejoró} \leftarrow o.\text{nuevoMejor} \vee f(x') < f(x)$ 
13   $o.\text{aceptado} \leftarrow o.\text{mejoró} \vee \text{ACEPTAR}(f(x'), f(x), T)$ 
14  si  $o.\text{aceptado}$  entonces  $x \leftarrow x'$ 
15  si  $o.\text{nuevoMejor}$  entonces  $x^* \leftarrow x'$ 
16   $\text{estanc} \leftarrow 0$  si  $o.\text{nuevoMejor}$ ; si no,  $\text{estanc} + 1$ 
17  si  $\text{estanc} \geq L_{\text{est}}$  entonces ▷ reinicio por estancamiento
18     $x \leftarrow x^*$ ;  $T \leftarrow \rho T_0$ ;  $\text{estanc} \leftarrow 0$ 
19   $\text{APRENDER}(t, o)$ 
20   $T \leftarrow cT$  ▷ enfriamiento geométrico
21  si  $\text{fase1}$  entonces ▷ control de la fase de rutas
22     $b \leftarrow |\mathcal{U}(x^*)|$ 
23     $\text{sinBanco} \leftarrow 0$  si  $b < b_{\text{ant}}$ ; si no,  $\text{sinBanco} + 1$ 
24    si  $b = 0$  entonces ▷ ruta eliminada: se intenta con la siguiente
25       $\bar{x} \leftarrow x^*$ ; INICIARETAPA( $t$ )
26    si no, si  $t \geq T_{\text{máx}}$  o ( $b \geq B$  y  $\text{sinBanco} \geq L_{\text{rut}}$ ) entonces ▷ se abandona el intento
27       $\text{fase1} \leftarrow \text{falso}$ ; INICIARETAPA( $t$ )
28   $t \leftarrow t + 1$ 
29 devolver  $x^*$ 
```

Algoritmo 5: INICIARETAPA (función local startStage de ALNS::solve)

Entrada : iteración t en la que comienza la etapa

```
1  $\text{fase1} \leftarrow \text{fase1} \wedge t < T_{\text{máx}} \wedge K(\bar{x}) > K_{\text{mín}}$ 
2 si  $\neg \text{fase1}$  entonces ▷ comienza la fase 2
3    $T_{\text{fin}} \leftarrow t + T_{\text{máx}}$ 
4    $\text{ALINICIARFASE2}(t)$  ▷ aviso a la subclase (onDistancePhase)
5   $x \leftarrow \bar{x}$ 
6  si  $\text{fase1}$  entonces ▷ fase 1: una ruta menos
7    eliminar de  $\mathcal{R}(x)$  las rutas vacías
8     $\pi \leftarrow$  ruta de  $x$  con menos clientes
9    mover todos los clientes de  $\pi$  a  $\mathcal{U}(x)$ ; eliminar  $\pi$  de  $\mathcal{R}(x)$ 
10  $x^* \leftarrow x$ ;  $T \leftarrow T_0$ ;  $\text{estanc} \leftarrow 0$ ;  $\text{sinBanco} \leftarrow 0$ 
```

Tabla 2: Constantes del bucle común (`alns.cpp`, `operators.cpp`, `solution.cpp`).

Constante	Símbolo	Valor	Función
UNASSIGNED_COST	–	10^6	Peso de un cliente sin asignar en f
VEHICLE_COST	–	10^4	Peso de un vehículo en f
TEMP_SCALE	w	0.2	Empeoramiento relativo aceptado con probabilidad 0.5 en T_0
COOLING_RATE	c	0.9995	Factor de enfriamiento por iteración
STAGNATION_LIMIT	L_{est}	1500	Iteraciones sin nuevo mejor que disparan el reinicio
REHEAT_FACTOR	ρ	0.5	Fracción de T_0 a la que se recalienta
ROUTE_PHASE_BANK	B	5	Clientes sin asignar a partir de los cuales puede abandonarse un intento de la fase 1
ROUTE_PHASE_STALL	L_{rut}	2000	Iteraciones sin reducir los clientes sin asignar que disparan ese abandono
Q_MIN_RATIO / Q_MAX_RATIO	–	0.10 / 0.40	Fracción mínima y máxima de clientes extraídos
Q_MIN_CAP / Q_MAX_CAP	–	30 / 60	Topes absolutos de esas dos cotas
Q_MIN_FLOOR	–	4	Cota inferior absoluta de q
NOISE_PROB	–	0.5	Probabilidad de que la reparación use costos con ruido

3.1. Las dos fases de la búsqueda

Flota fija. Los operadores de reparación *nunca abren rutas nuevas*: insertan los clientes pendientes en las rutas que ya existen en la solución y, si alguno no cabe en ninguna, lo dejan en $\mathcal{U}$ (Sección 7). El número de rutas de una solución solo puede cambiar, por tanto, cuando el bucle común retira una de forma explícita. Esta es la diferencia de fondo con un ALNS de una sola fase, en el que la reparación abre rutas a demanda y la reducción de flota se confía a la función de costo: aquí la flota es un dato de cada etapa y la búsqueda debe encajar a todos los clientes en ella.

Fase 1: minimización de rutas. Se compone de una sucesión de *etapas*. Cada etapa parte de la última solución completa $\bar{x}$, le quita la ruta con menos clientes y deja esos clientes sin asignar (Algoritmo 5). La lista $\mathcal{U}$ actúa como el *banco de clientes* de [4]: los clientes pendientes viajan con la solución de una iteración a la siguiente y cada reparación intenta colocarlos junto con los q recién extraídos. Como f penaliza con 10^6 cada cliente sin asignar, el ALNS ordinario, sin ninguna modificación, trabaja para vaciar el banco. La etapa termina de una de estas tres formas:

- **Éxito.** La mejor solución de la etapa queda completa ($|\mathcal{U}(x^*)| = 0$): se ha eliminado una ruta. Esa solución pasa a ser $\bar{x}$ y comienza otra etapa, con una ruta menos.
- **Abandono por estancamiento.** Quedan al menos $B = 5$ clientes sin asignar en la mejor solución y han pasado $L_{\text{rut}} = 2000$ iteraciones sin que ese número disminuya. Es el criterio de corte de [4]: si el banco es grande y no se reduce, la ruta no es eliminable con un esfuerzo razonable. Con menos de 5 clientes pendientes el intento no se abandona por esta vía y continúa hasta agotar el presupuesto.
- **Presupuesto agotado.** Se alcanza la iteración $T_{\text{máx}}$.

En los dos últimos casos el intento en curso se descarta y la fase 1 termina. También termina, sin iniciar una etapa nueva, cuando $\bar{x}$ alcanza la cota inferior trivial de vehículos

$$K_{\text{mín}} = \left\lceil \frac{1}{Q} \sum_{i=1}^n \mu_i \right\rceil, \quad (5)$$

ya que con menos rutas no hay capacidad para toda la demanda. Si la solución inicial ya cumple $K(x_0) = K_{\min}$, la fase 1 no se ejecuta.

Fase 2: minimización de distancia. Parte de $\bar{x}$, la solución completa con menos rutas que encontró la fase 1 (no del intento fallido), y ejecuta siempre $T_{\max}$ iteraciones: si la fase 1 terminó en la iteración $t_1 \leq T_{\max}$, la búsqueda acaba en $T_{\text{fin}} = t_1 + T_{\max}$. La flota sigue fija, de modo que el número de vehículos no puede superar el número de rutas de $\bar{x}$; sí puede bajar si una destrucción vacía una ruta y la reparación no vuelve a usarla, lo que f premia con 10^4 . Una candidata incompleta cuesta al menos 10^6 más que la solución actual, por lo que en esta fase es rechazada en la práctica (Sección 3.3) y nunca es un nuevo mejor: la solución devuelta es completa.

Reinicio de la búsqueda en cada etapa. Al comenzar cada etapa de la fase 1, y al comenzar la fase 2, se restablecen x , x^* , la temperatura ($T \leftarrow T_0$) y los contadores de estancamiento. En cambio, el estado del mecanismo de selección (los pesos de las ruletas o la tabla Q) *se conserva* entre etapas y entre fases; el único aviso que reciben las subclases es ALINICIARFASE2, que el método clásico ignora y el Q-learning aprovecha para volver a explorar (Sección 5.4). Obsérvese que dentro de una etapa de la fase 1 x^* es la mejor solución *de esa etapa*, normalmente incompleta; la mejor solución completa se guarda aparte en $\bar{x}$.

3.2. Ganancias relativas

El resultado o de cada iteración incluye dos ganancias, respecto de la mejor solución y de la actual, calculadas *antes* de actualizar x y x^* . La ganancia de una candidata x' sobre una referencia y es la mejora relativa en el *primer criterio en que difieren*, en el mismo orden que impone f (`relativeGain`):

$$G(y, x') = \begin{cases} \max\left(0, \frac{|\mathcal{U}(y)| - |\mathcal{U}(x')|}{|\mathcal{U}(y)|}\right) & \text{si } |\mathcal{U}(x')| \neq |\mathcal{U}(y)|, \\ \max\left(0, \frac{K(y) - K(x')}{K(y)}\right) & \text{si no, y } K(x') \neq K(y), \\ \max\left(0, \frac{D(y) - D(x')}{D(y)}\right) & \text{en otro caso.} \end{cases} \quad (6)$$

Cada mejora se expresa así en la escala de su propio criterio: colocar un cliente del banco vale $1/|\mathcal{U}|$, ahorrar un vehículo vale $1/K$ y una reducción de distancia vale $\Delta D/D$. Si la ganancia se midiera sobre f , las mejoras de distancia quedarían diluidas por los términos $10^4 K$ y $10^6 |\mathcal{U}|$, que no cambian. Una candidata que empeora el primer criterio en que difiere tiene ganancia 0 aunque mejore los siguientes. Las ganancias solo las utiliza el Q-learning (Sección 5.2); el método clásico usa únicamente las banderas de o .

3.3. Criterio de aceptación

Se usa el criterio de recocido simulado [2]. Una candidata que no empeora la solución actual se acepta siempre; una que la empeora en $\Delta = f(x') - f(x) > 0$ se acepta con probabilidad

$$P(\text{aceptar}) = \exp(-\Delta/T). \quad (7)$$

Obsérvese que una candidata de igual costo ($\Delta = 0$) se acepta pero no se considera mejora, lo que permite desplazamientos laterales por mesetas de f sin que cuenten como éxito para el mecanismo de aprendizaje.

Las candidatas incompletas no reciben un tratamiento especial: se evalúan con f como cualquier otra. En la fase 1 son la norma, y el orden que induce f hace que la búsqueda prefiera,

por este orden, menos clientes pendientes, menos vehículos y menos distancia. Dejar un cliente más sin asignar supone $\Delta \geq 10^6$ menos lo que se ahorre en los otros términos, y su probabilidad de aceptación es despreciable a las temperaturas de trabajo; en la práctica, el banco de la solución actual no crece. A igualdad de clientes pendientes, en cambio, el recocido acepta empeoramientos de distancia con normalidad, y es ese movimiento el que reordena las rutas hasta que los clientes del banco encuentran hueco.

3.4. Esquema de temperatura

Temperatura inicial. T_0 se calibra una sola vez, respecto de la distancia de la solución inicial, de forma que un empeoramiento de $w D(x_0)$ se acepte con probabilidad 0.5:

$$T_0 = \frac{-w D(x_0)}{\ln 0.5} = \frac{0.2}{\ln 2} D(x_0) \approx 0.2885 D(x_0). \quad (8)$$

Con esta definición, la probabilidad de aceptar un empeoramiento Δ a temperatura T_0 es $2^{-\Delta/(wD(x_0))}$. Todas las etapas de la fase 1 y la fase 2 arrancan con esta misma T_0 . Conviene notar que T_0 escala con la distancia de la instancia, mientras que las penalizaciones de (4) son fijas: la probabilidad de aceptar a T_0 una solución con un vehículo adicional (reutilizando una ruta que había quedado vacía) es $2^{-5 \cdot 10^4 / D(x_0)}$, que es despreciable cuando $D(x_0)$ es del orden de unos pocos miles y deja de serlo cuando $D(x_0)$ es del orden de decenas de miles. La de aceptar un cliente más sin asignar es $2^{-5 \cdot 10^6 / D(x_0)}$, despreciable en todos los tamaños de instancia considerados.

Enfriamiento. Tras cada iteración $T \leftarrow cT$ con $c = 0.9995$. La temperatura se reduce a la mitad cada $\ln 0.5 / \ln c \approx 1386$ iteraciones y, sin recalentamientos, al cabo de las 25 000 iteraciones de la fase 2 valdría $c^{25000} T_0 \approx 3.7 \cdot 10^{-6} T_0$, es decir, un descenso prácticamente puro.

Reinicio por estancamiento. Si transcurren $L_{\text{est}} = 1500$ iteraciones consecutivas sin un nuevo mejor, la búsqueda vuelve a la mejor solución conocida ($x \leftarrow x^*$) y la temperatura se fija en $\rho T_0 = 0.5 T_0$. El reinicio combina dos efectos: *intensificación*, porque descarta la deriva acumulada y retoma el mejor punto conocido, y *diversificación*, porque el recalentamiento vuelve a permitir empeoramientos. Una consecuencia cuantitativa es que, en un periodo prolongado de estancamiento, la temperatura oscila de forma periódica dentro del intervalo

$$[\rho c^{L_{\text{est}}} T_0, \rho T_0] \approx [0.236 T_0, 0.5 T_0],$$

y nunca llega a congelarse. La temperatura solo desciende por debajo de esa banda mientras se sigan encontrando nuevos mejores con una separación menor que L_{est} iteraciones. El mecanismo actúa igual en las dos fases. En la fase 1, «nuevo mejor» es cualquier descenso de f dentro de la etapa, incluidas las mejoras de distancia a igualdad de banco; por eso el abandono de un intento no se decide con este contador sino con uno propio (*sinBanco*), que solo atiende al número de clientes pendientes.

El contador de estancamiento se actualiza *antes* de APRENDER, pero el resultado o que recibe el mecanismo de aprendizaje es el de la iteración recién evaluada y no se ve alterado por el reinicio. Del mismo modo, el cambio de etapa o de fase se decide *después* de APRENDER, de manera que la iteración que completa una solución en la fase 1 se aprende como cualquier otra.

3.5. Catálogo de operadores y grado de destrucción

El Algoritmo 6 muestra cómo se aplica el par elegido. Hay 6 operadores de destrucción y 5 de reparación (Tabla 3), es decir, 30 pares posibles.

Algoritmo 6: APLICAROPERADORES (`applyOperators`)

Entrada : solución x' , índice de destrucción $j \in \{0, \dots, 5\}$, índice de reparación $k \in \{0, \dots, 4\}$

- 1 $q_{\min} \leftarrow \max(4, \min(30, \lfloor 0.10n \rfloor))$
- 2 $q_{\max} \leftarrow \max(q_{\min} + 1, \min(60, \lfloor 0.40n \rfloor))$
- 3 $q \sim U\{q_{\min}, \dots, q_{\max}\}$
- 4 $des_j(x', q)$
- 5 $ruido \leftarrow$ verdadero con probabilidad 0.5
- 6 $rep_k(x', ruido)$

Tabla 3: Catálogo de operadores, en el orden de sus índices en el código.

Índice	Operador	Criterio
<i>Destrucción</i>		
0	<code>randomRemoval</code>	Clientes al azar
1	<code>routeRemoval</code>	Rutas completas al azar
2	<code>worstRemoval</code>	Clientes cuyo retiro ahorra más distancia (aleatorizado)
3	<code>shawRemoval</code>	Clientes parecidos entre sí en espacio, tiempo y demanda
4	<code>clusterRemoval</code>	Uno de los dos grupos geográficos de una ruta, y así en rutas vecinas
5	<code>timeWindowRemoval</code>	Clientes con las ventanas de tiempo más estrechas
<i>Reparación</i> (cada una, con o sin ruido en los costos)		
0	<code>greedyInsertion</code>	Inserción de menor costo
1	<code>regret2Insertion</code>	Mayor arrepentimiento entre las dos mejores posiciones
2	<code>regret3Insertion</code>	Mayor arrepentimiento entre las tres mejores rutas
3	<code>regret4Insertion</code>	Mayor arrepentimiento entre las cuatro mejores rutas
4	<code>regretMInsertion</code>	Mayor arrepentimiento sobre todas las rutas

Grado de destrucción. Se sortea de forma uniforme entre el 10 % y el 40 % de los clientes, con los topes absolutos de Pisinger y Ropke [3]:

$$q \in [\min(0.1n, 30), \min(0.4n, 60)],$$

es decir, $q \in [10, 40]$ para $n = 100$, $q \in [20, 60]$ para $n = 200$ y $q \in [30, 60]$ para todo $n \geq 300$. En las instancias de 400, 600 y 800 clientes el vecindario tiene, por tanto, el mismo tamaño absoluto y representa una fracción decreciente de la solución (entre el 3.75 % y el 7.5 % para $n = 800$). El tope mantiene acotado el costo de una iteración, que está dominado por la reparación (Sección 7), y es lo que hace asumible un presupuesto de hasta $2T_{\max} = 50\,000$ iteraciones en las instancias grandes. En la fase 1 la reparación debe reinsertar, además de los q clientes extraídos, los que ya estaban en el banco.

Ruido. En cada iteración se decide con probabilidad 0.5, de forma independiente del par elegido, si la reparación trabaja con los costos de inserción exactos o con costos perturbados (Sección 7.3). El ruido no es un operador aparte ni una decisión del mecanismo de aprendizaje: cada operador de reparación se aplica, en promedio, la mitad de las veces en cada modalidad, y el crédito que recibe mezcla ambas.

4. ALNS clásico: selección por ruleta adaptativa

`ALNS_Classic` implementa el mecanismo adaptativo de Ropke y Pisinger [4]. Mantiene dos ruletas *independientes*, una para los 6 operadores de destrucción y otra para los 5 de reparación. Cada ruleta guarda, para cada operador i , un peso w_i (inicialmente 1), un puntaje acumulado S_i y un contador de usos u_i dentro del segmento en curso.

4.1. Selección

En cada iteración se sortea un operador de cada ruleta con probabilidad proporcional a su peso:

$$P(i) = \frac{w_i}{\sum_l w_l}. \quad (9)$$

Como los dos sorteos son independientes, la probabilidad de un par es el producto $P^-(j) P^+(k)$: el método no puede representar que un operador de destrucción funcione mejor con un operador de reparación concreto.

4.2. Asignación de crédito y actualización

Tras evaluar la candidata, ambos operadores del par reciben el mismo puntaje σ , que depende únicamente de la *categoría* del resultado y no de su magnitud:

$$\sigma = \begin{cases} 33 & \text{si la candidata es un nuevo mejor global,} \\ 13 & \text{si mejora a la solución actual,} \\ 9 & \text{si no mejora pero es aceptada,} \\ 0 & \text{si es rechazada.} \end{cases} \quad (10)$$

Los tres valores positivos son los propuestos en [4], asignados aquí en orden monótono con la calidad del resultado. Las categorías se determinan comparando valores de f , por lo que valen igual en las dos fases: en la fase 1, colocar un cliente del banco es una mejora (y normalmente un nuevo mejor de la etapa), y una candidata que deja más clientes sin asignar que la solución actual es, en la práctica, un rechazo. Los pesos no cambian en cada iteración sino al cierre de cada segmento de $\ell = 100$ iteraciones, con una media móvil exponencial sobre el puntaje medio del segmento:

$$w_i \leftarrow \max\left(w_{\min}, \lambda w_i + (1 - \lambda) \frac{S_i}{u_i}\right) \quad \text{si } u_i > 0, \quad \lambda = 0.9, \quad w_{\min} = 0.01. \quad (11)$$

Un operador que no se usó en el segmento conserva su peso. El Algoritmo 7 reúne ambas operaciones.

Algoritmo 7: Selección y aprendizaje del ALNS clásico (ALNS_Classic)

```

1 procedimiento SELECCIONAR( $t$ )
2    $j \leftarrow$  sorteo en la ruleta de destrucción según (9)
3    $k \leftarrow$  sorteo en la ruleta de reparación según (9)
4   devolver ( $j, k$ )
5 procedimiento APRENDER( $t, o$ )
6    $\sigma \leftarrow$  puntaje de  $o$  según (10)
7    $S_j^- \leftarrow S_j^- + \sigma; \quad u_j^- \leftarrow u_j^- + 1$ 
8    $S_k^+ \leftarrow S_k^+ + \sigma; \quad u_k^+ \leftarrow u_k^+ + 1$ 
9   si  $t \bmod 100 = 0$  entonces  $\triangleright$  cierre de segmento
10    para cada ruleta y cada operador  $i$  de la ruleta hacer
11      si  $u_i > 0$  entonces  $w_i \leftarrow \max(w_{\min}, \lambda w_i + (1 - \lambda) S_i / u_i)$ 
12       $S_i \leftarrow 0; \quad u_i \leftarrow 0$ 

```

4.3. Propiedades relevantes

- **Normalización por uso.** El peso se actualiza con el puntaje *medio* S_i/u_i , por lo que un operador elegido con frecuencia no se refuerza solo por haber sido usado más veces.

- **Inercia.** Con $\lambda = 0.9$ cada segmento aporta el 10 % del peso nuevo. El puntaje medio está acotado en $[0, 33]$, de modo que los pesos convergen hacia ese rango de forma gradual: partiendo de $w_i = 1$, hacen falta varios segmentos (varios cientos de iteraciones) para que la distribución se aleje de la uniforme. Los segmentos se cuentan sobre el índice global de iteración, sin distinguir etapas ni fases: en las 25 000 iteraciones de la fase 2 hay 250 actualizaciones de pesos, a las que se suman hasta otras 250 en la fase 1.
- **Continuidad entre fases.** El método no reacciona al cambio de fase (`onDistancePhase` conserva su implementación vacía): los pesos aprendidos mientras se eliminaban rutas son el punto de partida de la fase de distancia, y se readaptan al nuevo objetivo solo a través de la media móvil (11).
- **Exploración permanente.** La selección es proporcional, no voraz, y $w_{\min}$ impide que un peso se anule. Todos los operadores conservan una probabilidad no nula durante toda la búsqueda, y la distribución se adapta a la fase en curso (por ejemplo, los operadores que aceptan empeoramientos puntúan más cuando la temperatura es alta).
- **Crédito ordinal.** Una mejora mínima de distancia y la colocación de un cliente del banco reciben el mismo puntaje si pertenecen a la misma categoría; el método mide con qué frecuencia un operador tiene éxito, no cuánto mejora.
- **Sin contexto.** La distribución no depende del estado de la búsqueda más que a través de los pesos acumulados.

5. ALNS con Q-learning

ALNS_QLearning sustituye las dos ruletas por un agente de aprendizaje por refuerzo tabular que aplica Q-learning [9, 8]. La elección de operadores se modela como un proceso de decisión con los siguientes elementos.

- **Estados.** Dos estados, $z \in \{0, 1\}$: $z = 1$ si la iteración anterior produjo una mejora y $z = 0$ en caso contrario. El estado inicial es $z = 0$.
- **Acciones.** Cada acción es un *par* $\theta = (j, k)$ de operador de destrucción y de reparación, codificado como $\theta = 5j + k$. Hay $6 \times 5 = 30$ acciones.
- **Tabla.** $Q \in \mathbb{R}^{2 \times 30}$, inicializada en cero.
- **Memoria de mejoras.** Un vector $M \in \mathbb{R}^{30}$, inicializado en cero, con la mejora reciente de cada acción (`recent_improvement`); se usa para calcular el costo de oportunidad (Sección 5.2).

A diferencia del método clásico, la unidad de decisión es el par completo: el agente puede aprender que una destrucción funciona con una reparación y no con otra. A cambio debe estimar 60 valores en lugar de 11 pesos.

Tabla 4: Parámetros del Q-learning (`SolverParams`, valores por defecto). Todos pueden modificarse desde la línea de comandos con la sintaxis `clave=valor`. La última fila es una constante de `alns_qlearning.cpp`, no un parámetro.

Parámetro	Símbolo	Valor	Función
<code>alpha</code>	α	0.5	Tasa de aprendizaje
<code>gamma</code>	γ	0.7	Factor de descuento
<code>learning_loop</code>	T_{exp}	200	Iteraciones de exploración pura al inicio de la búsqueda y al inicio de la fase 2
<code>beta</code>	β	0.99	Decaimiento multiplicativo de ε por iteración
<code>epsilon_min</code>	$\varepsilon_{\text{mín}}$	0.01	Exploración residual
<code>eta</code>	η	0.8	Peso de la mejora global frente a la local en la recompensa
<code>reward_gain</code>	κ	10^4	Ganancia de la transformación logarítmica
<code>IMPROVEMENT_DECAY</code>	δ	0.9	Inercia de la memoria de mejoras M

5.1. Selección de la acción

La política es ε -voraz con un calendario de exploración en tres tramos (Algoritmo 8). Sea t_0 la iteración en la que comienza el calendario ($t_0 = 0$ al inicio de la búsqueda) y $t_{\text{exp}} = t_0 + T_{\text{exp}}$ la última iteración de exploración pura (`explore_until`):

$$\varepsilon_t = \begin{cases} 1 & t \leq t_{\text{exp}}, \\ \max(\varepsilon_{\text{mín}}, \beta^{t-t_{\text{exp}}-1}) & t > t_{\text{exp}}. \end{cases} \quad (12)$$

1. *Calentamiento* (las primeras 200 iteraciones): las acciones se eligen uniformemente al azar. Cada par se prueba en promedio $200/30 \approx 7$ veces y la tabla se actualiza con normalidad, de modo que al terminar ya contiene una primera estimación de todos los pares.
2. *Transición* (las 459 siguientes): ε decae geométricamente desde 1 y llega a $\varepsilon_{\text{mín}}$, pues $\lceil \ln 0.01 / \ln 0.99 \rceil = 459$.
3. *Explotación* (a partir de la iteración 660 del calendario, aproximadamente): $\varepsilon = 0.01$ y el 99 % de las decisiones son voraces.

El calendario se recorre *dos veces*: una desde el inicio de la búsqueda y otra desde el inicio de la fase 2 (Sección 5.4). En la fase 2, que dura $T_{\text{máx}} = 25\,000$ iteraciones, más del 97 % transcurre en explotación. La fase 1 tiene duración variable; si termina antes de unas 660 iteraciones, su calendario queda truncado y toda ella transcurre en calentamiento o transición.

En la rama voraz se elige la acción de mayor valor en el estado actual. Los empates (con tolerancia 10^{-12}) se resuelven al azar, lo que evita un sesgo sistemático hacia las acciones de índice bajo mientras la tabla conserva valores iguales.

Algoritmo 8: Selección ε -voraz (`ALNS_QLearning::selectOperators`)

Entrada : iteración t ; estado z ; tabla Q ; exploración actual ε ; fin de la exploración pura t_{exp}

Salida : par (j, k) ; se memoriza la acción θ

- 1 $\varepsilon_t \leftarrow 1$ si $t \leq t_{\text{exp}}$; si no, ε
 - 2 **si** $U(0, 1) < \varepsilon_t$ **entonces**
 - 3 $\theta \sim U\{0, \dots, 29\}$ $\triangleright$ *exploración*
 - 4 **si no**
 - 5 $Q^* \leftarrow \max_{\theta'} Q(z, \theta')$
 - 6 $\theta \leftarrow$ elemento al azar de $\{\theta' : Q(z, \theta') \geq Q^* - 10^{-12}\}$ $\triangleright$ *explotación*
 - 7 **devolver** ($\lfloor \theta/5 \rfloor, \theta \bmod 5$)
-

5.2. Función de recompensa

La recompensa se construye a partir de las dos ganancias relativas que calcula el bucle común (Sección 3.2), ambas medidas *antes* de actualizar x y x^* :

$$g_t^{\text{mej}} = G(x^*, x'), \quad g_t^{\text{act}} = G(x, x'),$$

con G definida en (6): la mejora relativa en el primer criterio (clientes sin asignar, vehículos o distancia) en que la candidata difiere de la referencia.

1. Transformación logarítmica. Cada ganancia se transforma con

$$\varphi(g) = \ln(1 + \kappa g) \quad \text{si } g > 0, \quad \varphi(g) = 0 \quad \text{si } g \leq 0. \quad (13)$$

Su papel se entiende a partir de las escalas de G . Una reducción de distancia ΔD produce $g = \Delta D/D$, un valor pequeño (por ejemplo, 10^{-3} al ahorrar 5 unidades sobre 5000); ahorrar un vehículo produce $g = 1/K$ y colocar c clientes del banco produce $g = c/|\mathcal{U}|$, valores uno o varios órdenes de magnitud mayores. Con $\kappa = 10^4$ se obtiene

$$\varphi = \ln\left(1 + 10^4 \frac{\Delta D}{D}\right), \quad \varphi = \ln\left(1 + \frac{10^4}{K}\right), \quad \varphi = \ln\left(1 + 10^4 \frac{c}{|\mathcal{U}|}\right),$$

respectivamente. La ganancia κ fija la resolución de la señal: las mejoras relativas menores que $1/\kappa = 10^{-4}$ quedan en el tramo casi lineal de φ y las mayores se comprimen. El logaritmo impide así que los avances en los criterios prioritarios sepulsen la señal de las mejoras de distancia, sin dejar de ordenarlos: por ejemplo, con $K = 20$ y $D = 5000$, ahorrar 10 unidades de distancia da $\varphi \approx 3.0$, eliminar un vehículo da $\varphi \approx 6.2$ y colocar uno de 8 clientes pendientes da $\varphi \approx 7.1$. Las ganancias por clientes colocados son las características de la fase 1; en la fase 2 predominan las de distancia.

2. Combinación global–local. Las dos ganancias transformadas se mezclan con peso $\eta = 0.8$:

$$I_t = \eta \varphi(g_t^{\text{mej}}) + (1 - \eta) \varphi(g_t^{\text{act}}). \quad (14)$$

Como x^* nunca es peor que x , una ganancia global positiva implica una ganancia local positiva; por tanto $I_t > 0$ si y solo si la candidata mejora a la solución actual. Una mejora solo local recibe el 20 % de su valor transformado; un nuevo mejor recibe además el 80 % de $\varphi(g_t^{\text{mej}})$. El agente es recompensado por cualquier progreso, pero sobre todo por superar el mejor conocido.

3. Escalado temporal. La recompensa se multiplica por la fracción de búsqueda transcurrida, $\tau_t = t/T_{\text{fin}} \in (0, 1]$, donde T_{fin} es la última iteración prevista en ese momento (Algoritmo 4). Las mejoras son abundantes al principio y escasas al final; el factor τ_t da más valor a las mejoras tardías, que son las difíciles, y reduce el peso de las tempranas, que casi cualquier operador consigue. Como $T_{\text{fin}} = 2T_{\text{máx}}$ durante la fase 1 y $T_{\text{fin}} = t_1 + T_{\text{máx}}$ durante la fase 2 (con t_1 la iteración en que acabó la fase 1), τ_t no supera 0.5 en la fase 1 y crece desde $t_1/(t_1 + T_{\text{máx}})$ hasta 1 en la fase 2.

4. Costo de oportunidad. El agente mantiene, para cada acción θ , una media móvil exponencial M_θ de la mejora I que ha producido en sus usos recientes, contando como 0 los usos sin mejora. Una iteración sin mejora no recibe recompensa nula sino negativa, igual a la mayor mejora reciente *de las demás acciones*:

$$r_t = \begin{cases} \tau_t I_t & \text{si } I_t > 0, \\ -\tau_t \Omega_t & \text{si } I_t = 0, \end{cases} \quad \text{con } \Omega_t = \max_{\theta' \neq \theta} M_{\theta'}. \quad (15)$$

Calculada la recompensa, se actualiza la memoria de la acción usada, haya o no mejora:

$$M_\theta \leftarrow \delta M_\theta + (1 - \delta) I_t, \quad \delta = 0.9. \quad (16)$$

La interpretación es que una iteración desperdiciada cuesta lo que se habría podido ganar con la mejor alternativa. M_θ estima la mejora *media por uso* de la acción θ sobre una ventana efectiva de unos $1/(1 - \delta) = 10$ usos, de modo que combina con qué frecuencia mejora y cuánto. De la definición se siguen tres propiedades:

- la penalización es *adaptativa*: sube cuando alguna otra acción está produciendo mejoras y se anula cuando ninguna lo hace, en cuyo caso fallar no cuesta nada;
- al excluir a la propia acción del máximo, un par no se ve penalizado por sus propios éxitos anteriores, sino solo por lo que ofrecen las alternativas;
- M_θ solo cambia cuando θ se usa, por lo que el valor de una acción poco elegida refleja la última vez que se probó y no decae mientras tanto.

Las candidatas rechazadas reciben esta misma penalización, igual que las aceptadas que no mejoran: a diferencia del método clásico, la aceptación sin mejora no se premia.

5.3. Actualización de la tabla

El siguiente estado es $z' = 1$ si $I_t > 0$ y $z' = 0$ en caso contrario, y se aplica la regla de Q-learning [9]:

$$Q(z, \theta) \leftarrow Q(z, \theta) + \alpha \left[r_t + \gamma \max_{\theta'} Q(z', \theta') - Q(z, \theta) \right]. \quad (17)$$

El Algoritmo 9 resume el paso de aprendizaje completo, que se ejecuta en *todas* las iteraciones de las dos fases, incluidas las de calentamiento.

Algoritmo 9: Aprendizaje por Q-learning (ALNS_QLearning::learn)

Entrada : iteración t ; resultado o ; estado z y acción θ de la iteración

- 1 $I \leftarrow \eta \varphi(o.g^{\text{mej}}) + (1 - \eta) \varphi(o.g^{\text{act}})$
 - 2 $\tau \leftarrow t/T_{\text{fin}}$
 - 3 **si** $I > 0$ **entonces**
 - 4 $r \leftarrow \tau I; \quad z' \leftarrow 1$
 - 5 **si no**
 - 6 $\Omega \leftarrow \max_{\theta' \neq \theta} M_{\theta'}$ $\triangleright$ costo de oportunidad
 - 7 $r \leftarrow -\tau \Omega; \quad z' \leftarrow 0$
 - 8 $M_\theta \leftarrow \delta M_\theta + (1 - \delta) I$
 - 9 $Q(z, \theta) \leftarrow Q(z, \theta) + \alpha [r + \gamma \max_{\theta'} Q(z', \theta') - Q(z, \theta)]$
 - 10 $z \leftarrow z'$
 - 11 **si** $t > t_{\text{exp}}$ **entonces** $\varepsilon \leftarrow \max(\varepsilon_{\text{mín}}, \beta \varepsilon)$
-

5.4. Reacción al cambio de fase

Al comenzar la fase 2 el objetivo efectivo de la búsqueda cambia: deja de consistir en colocar clientes pendientes y pasa a ser reducir la distancia de una solución completa. Los pares que mejor vaciaban el banco no tienen por qué ser los que mejor acortan rutas. El agente recibe el aviso ALINICIARFASE2 del bucle común (Algoritmo 5) y responde como indica el Algoritmo 10.

Algoritmo 10: Inicio de la fase 2 en el Q-learning (ALNS_QLearning::onDistancePhase)

Entrada : iteración t en la que comienza la fase 2

- 1 $M_\theta \leftarrow 0$ para toda acción θ $\triangleright$ se olvidan las mejoras de la fase 1
 - 2 $\varepsilon \leftarrow 1; \quad t_{\text{exp}} \leftarrow t + T_{\text{exp}}$ $\triangleright$ se vuelve a explorar
-

- **Memoria de mejoras a cero.** Las mejoras de la fase 1 están en la escala de los clientes colocados ($\varphi \approx 7.1$ en el ejemplo anterior, y hasta $\ln(1 + 10^4) \approx 9.2$ al colocar el último cliente pendiente) y no son alcanzables en la fase 2. Si se conservaran, el costo de oportunidad penalizaría cada iteración sin mejora con una magnitud que ningún par puede ya compensar.
- **Nuevo calendario de exploración.** Se repiten las 200 iteraciones uniformes y el decaimiento de ε de (12), ahora con t_0 igual a la iteración de inicio de la fase 2, para que todos los pares vuelvan a evaluarse con el nuevo objetivo.
- **La tabla Q y el estado se conservan.** Los valores aprendidos en la fase 1 son el punto de partida de la fase 2; con $\alpha = 0.5$ y la nueva exploración uniforme, se sobrescriben en pocas visitas a cada celda.

Las etapas sucesivas de la fase 1, en cambio, comparten objetivo y no provocan ningún reinicio del agente: la tabla, la memoria de mejoras y ε continúan de una etapa a la siguiente.

5.5. Propiedades relevantes

- **Memoria corta.** Con tasa constante $\alpha = 0.5$, el valor $Q(z, \theta)$ es una media con pesos $2^{-1}, 2^{-2}, \dots$ sobre los objetivos de las últimas visitas a esa celda: las dos visitas más recientes concentran el 75 % del valor. La estimación reacciona de inmediato a los cambios de régimen de la búsqueda, lo cual es deseable porque el problema es no estacionario (la utilidad de un operador depende de la fase, de la temperatura y de la calidad de la solución actual), pero también es ruidosa.
- **Recompensa no estacionaria por diseño.** El factor τ_t hace que la escala de las recompensas, positivas y negativas, aumente a lo largo de cada fase, y el costo de oportunidad Ω_t sube y baja con el rendimiento reciente de los pares; una tasa de aprendizaje alta es coherente con esa elección.
- **Premios y penalizaciones en la misma escala.** La penalización de una iteración sin mejora es una mejora *media por uso*, no la mayor mejora observada. Para un par que mejora con frecuencia p y mejora media $\bar{I}$ cuando lo hace, la recompensa esperada de (15) es aproximadamente $\tau [p \bar{I} - (1 - p) \Omega]$, mientras que su propia memoria tiende a $M_\theta \approx p \bar{I}$. En consecuencia, la recompensa esperada de un par es positiva cuando su mejora media por uso supera, aproximadamente, a la de la mejor alternativa, y negativa en caso contrario: la señal compara cada par con el mejor de los demás, combinando frecuencia y magnitud de las mejoras. Esta es una consecuencia analítica de la fórmula; puede contrastarse empíricamente registrando la evolución de la tabla Q , de M y de la frecuencia de uso de cada par.
- **Papel del estado.** El estado distingue la situación «se acaba de mejorar» de «no se acaba de mejorar». Puesto que las mejoras son minoritarias, $z = 0$ es el estado dominante y su fila es la que se actualiza con más frecuencia; la fila $z = 1$ captura qué par conviene aplicar inmediatamente después de un éxito (por ejemplo, insistir con un operador intensivo). El término $\gamma \max_{\theta'} Q(z', \theta')$ propaga a cada par el valor de la situación a la que conduce.
- **Concentración.** Con $\varepsilon_{\min} = 0.01$, el método concentra casi todas las iteraciones en el par mejor valorado de cada estado, y cambia de par cuando las penalizaciones acumuladas lo hacen caer por debajo de otro. El reparto de uso entre operadores es, por tanto, mucho más desigual y más cambiante que el de la ruleta, que reparte en proporción a los pesos.

6. Operadores de destrucción

Los seis operadores tienen la firma $des(x, q)$: extraen aproximadamente q clientes de las rutas de x y los añaden a $\mathcal{U}(x)$, que en la fase 1 puede contener ya clientes pendientes de iteraciones

anteriores (el banco de la Sección 3.1). Ninguno elimina rutas: una ruta que se queda sin clientes permanece en la solución como ruta vacía. Todos terminan actualizando $K(x)$ y $D(x)$. Como salvaguarda, los bucles de las extracciones aleatoria, de rutas, de peores y de Shaw se detienen si $|\mathcal{U}|$ alcanza $n - 1$.

Se denomina *ruta activa* a la que tiene al menos un cliente, y *costo de retiro* de un cliente π_k al ahorro de distancia que produce quitarlo de su ruta:

$$\text{cr}(\pi_k) = d_{\pi_{k-1}\pi_k} + d_{\pi_k\pi_{k+1}} - d_{\pi_{k-1}\pi_{k+1}} \geq 0. \quad (18)$$

6.1. Extracción aleatoria

Algoritmo 11: EXTRACCIÓN ALEATORIA (`randomRemoval`)

Entrada : solución x , grado q

- 1 **mientras** *se hayan extraído menos de q clientes y existan rutas activas* **hacer**
 - 2 elegir una ruta activa π uniformemente al azar
 - 3 elegir una posición $k \sim \text{U}\{1, \dots, L - 2\}$ de π
 - 4 mover π_k a $\mathcal{U}$; `RECALCULARRUTA`(π)
-

El sorteo es en dos etapas (primero la ruta, luego el cliente), por lo que no es uniforme sobre los clientes: un cliente de una ruta con $L - 2$ clientes se elige con probabilidad $1/(m_{\text{act}}(L - 2))$, donde m_{act} es el número de rutas activas, y los clientes de rutas cortas son más propensos a salir. Es el operador de diversificación pura y su costo es $\mathcal{O}(q(m + \bar{L}))$.

6.2. Extracción de rutas

Algoritmo 12: EXTRACCIÓN DE RUTAS (`routeRemoval`)

Entrada : solución x , grado q

- 1 **mientras** *se hayan extraído menos de q clientes y existan rutas activas* **hacer**
 - 2 elegir una ruta activa π uniformemente al azar
 - 3 mover todos los clientes de π a $\mathcal{U}$; $\pi \leftarrow (0, 0)$
-

Vacía rutas enteras hasta alcanzar q , por lo que puede superar q en hasta una ruta. La ruta vaciada permanece en la solución como ruta vacía: no cuenta como vehículo, pero queda disponible para la reparación. El operador obliga a redistribuir por completo los clientes de varios vehículos. Como la reparación no abre rutas, es además el principal mecanismo por el que la fase 2 puede terminar con menos vehículos que los heredados de la fase 1: basta con que los clientes de una ruta vaciada se recolquen en las demás.

6.3. Extracción de los peores

Extrae clientes con costo de retiro alto, es decir, los que están «mal colocados». La elección se aleatoriza como en [4]: con la lista $\mathcal{L}$ de clientes ordenada por costo de retiro decreciente, se toma el elemento de índice

$$\iota = \lfloor y^p |\mathcal{L}| \rfloor, \quad y \sim \text{U}(0, 1), \quad p = 3. \quad (19)$$

Como $P(\iota < k) = (k/|\mathcal{L}|)^{1/p}$, con $p = 3$ el 10 % superior de la lista se elige con probabilidad 0.46 y el 1 % superior con probabilidad 0.22: hay un sesgo fuerte hacia los peores sin llegar a ser determinista.

Optimización: lista ordenada incremental. La formulación directa recalcula y ordena los costos de todos los clientes antes de cada extracción, con costo $\mathcal{O}(qn \log n)$. Pero retirar un cliente solo cambia el costo de retiro de sus dos vecinos en la ruta. El Algoritmo 13 ordena una sola vez y después mantiene la lista: el cliente extraído y sus vecinos se localizan por búsqueda binaria, usando el orden total (costo decreciente, identificador creciente), y se reubican. Un mapa auxiliar cliente $\rightarrow$ ruta evita buscar al cliente en toda la solución.

Algoritmo 13: EXTRACCIÓNDEPEORES (`worstRemoval`)

Entrada : solución x , grado q

- 1 construir $rutaDe[\cdot]$ (cliente $\rightarrow$ índice de ruta)
- 2 calcular $cr(i)$ para todo cliente en ruta según (18)
- 3 $\mathcal{L} \leftarrow$ clientes ordenados por (cr decreciente, identificador creciente)
- 4 **mientras** se hayan extraído menos de q clientes y $\mathcal{L} \neq \emptyset$ **hacer**
- 5 $y \sim U(0, 1)$; $\iota \leftarrow \min(\lfloor y^p |\mathcal{L}| \rfloor, |\mathcal{L}| - 1)$
- 6 **si** $\mathcal{L}[\iota]$ empata en costo con $\mathcal{L}[\iota - 1]$ o con $\mathcal{L}[\iota + 1]$ **entonces**
- 7 $i \leftarrow \text{REFERENCIAPEORES}(x, \iota)$ $\triangleright$ ordenamiento completo original
- 8 **si no** $i \leftarrow \mathcal{L}[\iota]$
- 9 $\pi \leftarrow$ ruta $rutaDe[i]$; $k \leftarrow$ posición de i en π
- 10 quitar i de $\mathcal{L}$ por búsqueda binaria; $rutaDe[i] \leftarrow -1$
- 11 mover i a $\mathcal{U}$; $\text{RECALCULARRUTA}(\pi)$
- 12 **para cada** vecino $v \in \{\pi_{k-1}, \pi_k\}$ con $v \neq 0$ **hacer**
- 13 $c' \leftarrow cr(v)$ en la ruta modificada
- 14 **si** $c' \neq cr(v)$ anterior **entonces** reubicar v en $\mathcal{L}$ por búsqueda binaria con costo c'

Equivalencia con la versión de referencia. Cuando el elemento elegido empata en costo con alguno de sus vecinos en la lista, el orden entre ellos depende del algoritmo de ordenamiento; en ese caso el operador delega la elección en el procedimiento original (`worstByFullSort`), que reconstruye y ordena la lista completa. Cuando no hay empate, el elemento de índice ι es único y coincide en ambas versiones. De este modo la versión optimizada elige siempre el mismo cliente que la original y, para una misma semilla, la trayectoria de búsqueda es idéntica: la optimización reduce el tiempo de cómputo sin alterar los resultados. El mismo principio se aplica en la extracción de Shaw y en los operadores de reparación.

El costo pasa de $\mathcal{O}(qn \log n)$ a $\mathcal{O}(n \log n)$ para el ordenamiento inicial más, por extracción, tres búsquedas binarias, el desplazamiento contiguo de elementos de la lista y un recálculo de ruta $\mathcal{O}(\bar{L})$.

6.4. Extracción de Shaw

Propuesta por Shaw [6], extrae clientes *parecidos* entre sí, con la idea de que son intercambiables y la reparación podrá recolocarlos de otra manera. La afinidad entre dos clientes combina distancia, diferencia de apertura de ventana y diferencia de demanda (menor valor significa más afinidad):

$$\text{Rel}(i, j) = 9 d_{ij} + 3 |a_i - a_j| + 2 |\mu_i - \mu_j|. \quad (20)$$

Los pesos (9, 3, 2) son los de [4]; los términos no se normalizan, por lo que su influencia relativa depende de las escalas de la instancia. El primer cliente se elige al azar (ruta y posición). Después, en cada paso, se toma como base un cliente al azar entre los ya extraídos y se extrae uno de los que siguen en ruta, elegido entre los más afines a la base con el mismo sesgo (19) ($p = 3$).

Optimización: orden de afinidad precalculado. $\text{Rel}(i, j)$ depende solo de la instancia, no de la solución. Por ello el orden de afinidad de cada cliente respecto de todos los demás puede calcularse una única vez. La primera invocación del operador construye la tabla $orden[b]$

(`Instance::shaw_order`), que contiene para cada cliente b los otros $n - 1$ clientes ordenados por (afinidad creciente, identificador creciente). El costo es $\mathcal{O}(n^2 \log n)$ una sola vez por ejecución y la memoria $n(n - 1)$ enteros (unos 2.6 MB para $n = 800$); la tabla pertenece a la instancia y la comparten todas las soluciones.

A partir de ahí no se ordena nada: el Algoritmo 14 recorre la fila de la base saltando los clientes que ya no están en ruta hasta alcanzar el rango deseado. Como el sesgo concentra ι en valores pequeños, el recorrido suele ser corto.

Algoritmo 14: EXTRACCIÓNDESHAW (`shawRemoval`)

Entrada : solución x , grado q

```

1  elegir al azar una ruta activa y un cliente  $i_0$  de ella; mover  $i_0$  a  $\mathcal{U}$ 
2   $\mathcal{E} \leftarrow \{i_0\}$   $\triangleright$  clientes extraídos por este operador
3  construir  $rutaDe[\cdot]$ ;  $enRuta \leftarrow$  número de clientes en ruta
4  mientras  $|\mathcal{E}| < q$  y  $enRuta > 0$  hacer
5       $b \leftarrow$  elemento al azar de  $\mathcal{E}$ 
6       $y \sim U(0, 1)$ ;  $\iota \leftarrow \min(\lfloor y^p enRuta \rfloor, enRuta - 1)$ 
7      recorrer  $orden[b]$  saltando los clientes con  $rutaDe = -1$  hasta obtener los de rango  $\iota - 1, \iota$  e  $\iota + 1$ 
8       $i \leftarrow$  cliente de rango  $\iota$ 
9      si  $Rel(b, i)$  empata con la del rango  $\iota - 1$  o la del rango  $\iota + 1$  entonces
10          $i \leftarrow REFERENCIA\text{SHAW}(x, b, \iota)$   $\triangleright$  ordenamiento completo original
11         mover  $i$  a  $\mathcal{U}$  y a  $\mathcal{E}$ ;  $RECALCULARRUTA(ruta\ de\ i)$ 
12          $rutaDe[i] \leftarrow -1$ ;  $enRuta \leftarrow enRuta - 1$ 
```

El costo por extracción pasa de $\mathcal{O}(n \log n)$ (calcular y ordenar las afinidades de todos los clientes en ruta) a un recorrido lineal en el peor caso y típicamente mucho más corto. El tratamiento de empates es el mismo que en la Sección 6.3 y garantiza la equivalencia con la versión de referencia.

6.5. Extracción de grupos

Es la extracción de *clusters* de Pisinger y Ropke [3]. Parte de la observación de que, al extraer clientes parecidos de una ruta, la reparación tiende a devolverlos a ella si el resto de la ruta sigue cerca. El operador divide los clientes de una ruta en dos grupos geográficos y extrae uno *entero*, y repite el proceso en rutas vecinas hasta alcanzar q (Algoritmo 15).

Los dos grupos se obtienen con el algoritmo de Kruskal sobre el grafo completo de los clientes de la ruta, con las distancias d_{ij} como pesos, *detenido cuando quedan dos componentes conexas*: las aristas se recorren de menor a mayor y cada una fusiona las componentes de sus extremos si son distintas (estructura de conjuntos disjuntos con compresión de caminos). El resultado equivale a quitar la arista más larga del árbol de expansión mínima, es decir, a separar la ruta por su mayor salto espacial.

Algoritmo 15: EXTRACCIÓNDEGRUPOS (`clusterRemoval`)

Entrada : solución x , grado q

```
1  $\pi \leftarrow$  ruta activa elegida uniformemente al azar;  $\mathcal{E} \leftarrow \emptyset$   $\triangleright$  extraídos por este operador
2 mientras exista  $\pi$  hacer
3    $\mathcal{C} \leftarrow$  clientes de  $\pi$ 
4   ordenar los pares  $\{a, b\} \subseteq \mathcal{C}$  por  $d_{ab}$  creciente
5   fusionar componentes en ese orden hasta que queden 2  $\triangleright$  Kruskal truncado
6    $\mathcal{G} \leftarrow$  componente de un cliente de  $\mathcal{C}$  elegido uniformemente al azar
7   mover los clientes de  $\mathcal{G}$  a  $\mathcal{U}$  y a  $\mathcal{E}$ ; RECALCULARRUTA( $\pi$ )
8   si  $|\mathcal{E}| \geq q$  entonces salir del bucle
9    $s \leftarrow$  elemento al azar de  $\mathcal{E}$ 
10   $\pi \leftarrow$  ruta, distinta de la actual, que contiene al cliente en ruta más cercano a  $s$  (ninguna si no
    quedan clientes en otras rutas)
```

Conviene destacar tres detalles:

- **Elección del grupo.** El grupo se elige sorteando un cliente de la ruta, no una de las dos componentes, por lo que la probabilidad de extraer un grupo es proporcional a su tamaño. Si la ruta tiene uno o dos clientes, cada componente es un único cliente; con un solo cliente, la ruta queda vacía.
- **Encadenamiento.** La siguiente ruta es la del cliente más cercano a uno de los ya extraídos, lo que concentra la destrucción en una región del plano: los grupos extraídos son vecinos entre sí y pueden intercambiarse entre rutas adyacentes. Solo se excluye la ruta recién tratada, de modo que el proceso puede volver más adelante a una ruta ya dividida y dividir lo que quedó de ella.
- **Grado efectivo.** El operador extrae grupos completos, así que puede superar q en hasta un grupo.

Frente a la extracción de Shaw, que elige cliente a cliente por afinidad con sesgo aleatorio, aquí la unidad extraída es un bloque espacial compacto y contiguo de una ruta. El costo por ruta tratada es $\mathcal{O}(\bar{L}^2 \log \bar{L})$ para ordenar los pares, más $\mathcal{O}(n)$ para localizar la ruta siguiente.

6.6. Extracción por ventana de tiempo

Algoritmo 16: EXTRACCIÓNPORVENTANA (`timeWindowRemoval`)

Entrada : solución x , grado q

```
1  $\mathcal{C} \leftarrow$  clientes en ruta ordenados por anchura de ventana  $b_i - a_i$  creciente
2  $\mathcal{D} \leftarrow$  los  $\min(q, |\mathcal{C}|)$  primeros de  $\mathcal{C}$ 
3 ordenar  $\mathcal{D}$  por (ruta, posición) decreciente  $\triangleright$  mantiene válidos los índices al borrar
4 para cada  $i \in \mathcal{D}$  hacer
5    $\lfloor$  mover  $i$  a  $\mathcal{U}$ ; RECALCULARRUTA(ruta de  $i$ )
```

Es un operador específico del VRPTW. Los clientes con ventanas estrechas son los más difíciles de ubicar y los que condicionan la estructura temporal de las rutas; al extraerlos todos a la vez, la reparación puede redecidir su asignación de forma conjunta en lugar de heredar su posición. Es el único operador de destrucción determinista: para una solución y un q dados extrae siempre el mismo conjunto, de modo que su variabilidad proviene de q , del operador de reparación con que se empareje y del ruido de este. Su costo es $\mathcal{O}(n \log n)$.

7. Operadores de reparación

Los cinco operadores tienen la firma $rep(x, ruido)$ y reinsertan los clientes de $\mathcal{U}(x)$ uno a uno. Todos son heurísticas constructivas de inserción: en cada paso eligen un cliente y una posición según su criterio, ejecutan la inserción y repiten. Son instancias de una misma plantilla (Sección 7.2) y comparten las siguientes reglas.

- **Rutas admisibles.** Una ruta admite al cliente u si $carga + \mu_u \leq Q$. Las rutas vacías que haya dejado la destrucción son rutas admisibles como cualquier otra.
- **Costo de inserción.** Es el incremento de distancia Δ del Algoritmo 2, exacto o perturbado con ruido (Sección 7.3). Insertar en una ruta vacía cuesta $2d_{0u}$ y *no* incluye la penalización de 10^4 por vehículo: la reparación razona solo en distancia. El control del número de vehículos recae en la estructura en dos fases de la búsqueda y en la función de costo, a través del criterio de aceptación.
- **Flota fija.** La reparación no abre rutas nuevas. Si ningún cliente pendiente tiene una inserción factible en las rutas existentes, termina y deja esos clientes en $\mathcal{U}$. La candidata resultante es incompleta y el bucle común la evalúa con la penalización de 10^6 por cliente sin asignar (Sección 3.1).

7.1. Formulación directa y su costo

Sea $U = |\mathcal{U}|$ el número de clientes por insertar: $U = q$ tras una destrucción en la fase 2, y q más los clientes del banco en la fase 1. La formulación directa evalúa en cada paso todas las posiciones de todas las rutas para todos los clientes pendientes: $\mathcal{O}(Un)$ evaluaciones por paso y $\mathcal{O}(U^2n)$ en total. Aun con $q \leq 60$, esto supone del orden de 10^6 evaluaciones de inserción por iteración para $n = 800$, demasiado para un presupuesto de decenas de miles de iteraciones por corrida.

7.2. Inserción con caché por ruta

La observación clave es que una inserción modifica *una sola ruta*. Las evaluaciones de todos los demás pares (cliente, ruta) siguen siendo válidas. El Algoritmo 17 (**cachedInsertion**) mantiene una matriz $M[u][r]$ con un resumen de la ruta r para el cliente pendiente u :

$$M[u][r] = (mejor, segundo, pos),$$

el menor y el segundo menor costo de inserción factible de u en r y la posición del primero (∞ si no existen). Calcular una entrada cuesta $\mathcal{O}(L_r)$ gracias a la evaluación en tiempo constante. Tras cada inserción en la ruta r^* solo se recalcula la columna r^* .

Sobre cada fila se define un *agregador* $G[u]$ que la resume en lo que el criterio necesita: la ruta de destino y una *clave* de prioridad. Los cinco operadores difieren únicamente en el agregador (Sección 7.4).

Algoritmo 17: INSERCIÓNCONCACHÉ(G) (cachedInsertion<Agg>)

Entrada : solución x con $\mathcal{U} \neq \emptyset$; bandera *ruido*; tipo de agregador G

1 $\nu \leftarrow 0.025 d_{\max}$ si *ruido*; si no, $\nu \leftarrow 0$ $\triangleright$ amplitud del ruido

2 **para cada** $u \in \mathcal{U}$ **hacer**

3 **para cada** ruta r **hacer** $M[u][r] \leftarrow \text{ENTRADA}(u, r, \nu)$

4 $G[u].\text{RECONSTRUIR}(M[u])$

5 **mientras** $\mathcal{U} \neq \emptyset$ **hacer**

6 $u^* \leftarrow \arg \max\{G[u].\text{clave} : u \in \mathcal{U}, G[u] \text{ válido}\}$ $\triangleright$ primer máximo

7 **si no existe** u^* **entonces salir del bucle** $\triangleright$ flota fija: los restantes quedan sin asignar

8 $r^* \leftarrow G[u^*].\text{ruta}$

9 insertar u^* en r^* en la posición $M[u^*][r^*].\text{pos}$; $\text{RECALCULARRUTA}(r^*)$

10 quitar u^* de $\mathcal{U}$, de M y de G $\triangleright$ en $\mathcal{O}(1)$, intercambiando con el último

11 **para cada** $u \in \mathcal{U}$ **hacer** $\triangleright$ solo cambió la columna r^*

12 $e_{\text{ant}} \leftarrow M[u][r^*]$; $M[u][r^*] \leftarrow \text{ENTRADA}(u, r^*, \nu)$

13 **si** $M[u][r^*] \neq e_{\text{ant}}$ **entonces** $G[u].\text{ACTUALIZAR}(M[u], r^*, e_{\text{ant}})$

14 actualizar $K(x)$ y $D(x)$

Algoritmo 18: ENTRADA (computeEntry)

Entrada : cliente u , ruta r con secuencia π , amplitud de ruido ν

Salida : $e = (\text{mejor}, \text{segundo}, \text{pos})$

1 $e \leftarrow (\infty, \infty, -)$

2 **si** $\text{carga}_r + \mu_u > Q$ **entonces devolver** e

3 **para** $k \leftarrow 0$ **hasta** $L - 2$ **hacer**

4 $(\text{factible}, \Delta) \leftarrow \text{EVALUARINSERCIÓN}(u, \pi, k)$

5 **si** $\neg \text{factible}$ **entonces continuar**

6 **si** $\nu > 0$ **entonces** $\Delta \leftarrow \max(0, \Delta + \xi)$ con $\xi \sim U(-\nu, \nu)$

7 **si** $\Delta < e.\text{mejor}$ **entonces**

8 $e.\text{segundo} \leftarrow e.\text{mejor}$; $e.\text{mejor} \leftarrow \Delta$; $e.\text{pos} \leftarrow k + 1$

9 **si no, si** $\Delta < e.\text{segundo}$ **entonces**

10 $e.\text{segundo} \leftarrow \Delta$

11 **devolver** e

Costo. La inicialización cuesta $\mathcal{O}(Un)$ evaluaciones. Cada paso posterior recalcula una columna, $\mathcal{O}(U\bar{L})$, y elige el máximo en $\mathcal{O}(U)$. El total es

$$\mathcal{O}(Un + U^2 \bar{L}) \quad \text{frente a} \quad \mathcal{O}(U^2 n),$$

una reducción por un factor del orden del número de rutas $m = n/\bar{L}$ en el término dominante. A esto se suma la actualización de los agregadores, que se analiza a continuación.

7.3. Ruido en los costos de inserción

Cuando la bandera *ruido* está activa (lo está en la mitad de las iteraciones, Sección 3.5), cada costo de inserción factible se perturba con un término aditivo, como en [4]:

$$\tilde{\Delta} = \max(0, \Delta + \xi), \quad \xi \sim U(-\nu, \nu), \quad \nu = 0.025 d_{\max}, \quad (21)$$

donde $d_{\max}$ es la mayor distancia entre dos nodos de la instancia y 0.025 es la constante NOISE_RATE. La amplitud es absoluta, proporcional a la escala geográfica de la instancia y no al costo de cada inserción: altera el orden entre inserciones cuya diferencia es menor que 2ν y respeta las decisiones claras, de modo que dos reparaciones de la misma solución parcial pueden

producir resultados distintos. Es el mecanismo de diversificación de la reparación, y se aplica por igual a los cinco operadores.

El ruido se sortea al calcular cada entrada (Algoritmo 18), con un valor independiente por posición, y actúa *antes* de seleccionar el mejor y el segundo mejor costo de la ruta. En consecuencia afecta a las tres decisiones de la reparación: en qué posición de una ruta entraría un cliente, a qué ruta va y en qué orden se insertan los clientes. La entrada perturbada se conserva en la caché mientras su ruta no cambie y se vuelve a sortear cuando la columna se recalcula. La factibilidad no se perturba: una inserción elegida con ruido es siempre factible, y la distancia de la solución se calcula con los costos reales.

7.4. Agregadores y actualización perezosa

Reconstruir un agregador exige recorrer su fila, $\mathcal{O}(m)$. Hacerlo para todos los clientes en cada paso añadiría $\mathcal{O}(U^2m)$. Cada agregador implementa por ello una operación ACTUALIZAR que, conocida la entrada anterior y la nueva de la única columna modificada, decide en $\mathcal{O}(1)$ si el resumen puede haber cambiado y solo en ese caso reconstruye. Además, ACTUALIZAR ni siquiera se invoca cuando la entrada recalculada es idéntica a la anterior. Las condiciones de salto están escritas de modo que el resultado coincide siempre con el de una reconstrucción completa, incluidos los empates.

Inserción voraz (GreedyAgg). Resume la fila en el menor costo y su ruta:

$$c_u = \min_r M[u][r].mejor, \quad clave = -c_u.$$

Se inserta primero el cliente cuya mejor inserción es la más barata. Actualización: si la columna modificada es la ruta óptima actual, se reconstruye; si es otra, basta comparar su nuevo costo con c_u (y, en caso de empate, preferir la ruta de menor índice, que es lo que elegiría la reconstrucción). Es el criterio más miope: tiende a agotar pronto las inserciones baratas y a dejar para el final a los clientes difíciles.

Arrepentimiento 2 (Regret2Agg). Considera los dos menores costos de inserción del cliente sobre *todas las posiciones de todas las rutas*, $c_u^{(1)} \leq c_u^{(2)}$ (pueden pertenecer a la misma ruta; por eso cada entrada de M guarda también el segundo mejor costo de la ruta):

$$clave = c_u^{(2)} - c_u^{(1)}, \quad clave = 10^9 \text{ si solo existe una inserción factible.}$$

Se inserta primero el cliente que más perdería si no obtuviera su mejor posición; un cliente con una única posición factible tiene prioridad absoluta. Actualización: si la columna modificada no es la ruta óptima y tanto su mejor costo anterior como el nuevo superan $c_u^{(2)}$, ni $c_u^{(1)}$ ni $c_u^{(2)}$ pueden cambiar y se omite la reconstrucción.

Arrepentimiento k sobre rutas (RegretAgg<K>). Los tres operadores restantes comparten un agregador genérico, parametrizado por k . Los valores usados son $k = 3$ (**regret3Insertion**), $k = 4$ (**regret4Insertion**) y $k = m$, el número de rutas de la solución (**regretMInsertion**, que en el código corresponde a $K = 0$). Es el arrepentimiento de [4]: considera el mejor costo *por ruta* y ordena las rutas factibles, $c_u^{[1]} \leq c_u^{[2]} \leq c_u^{[3]} \leq \dots$. Con $h = \min(k, \text{número de rutas factibles})$,

$$clave = \sum_{l=2}^h (c_u^{[l]} - c_u^{[1]}) + 10^4 (k - h). \quad (22)$$

El primer término mide cuánto se perdería en total si el cliente tuviera que conformarse con su segunda, tercera, $\dots$, k -ésima mejor ruta. El segundo prioriza a los clientes con menos de k rutas

factibles, con una penalización de 10^4 por cada ruta que falta. La ruta de destino es la de menor costo (la de menor índice en caso de empate).

El agregador conserva, además de la clave y la ruta, el k -ésimo menor costo $c_u^{[k]}$ (o ∞ si hay menos de k rutas factibles). Actualización: si el mejor costo de la columna modificada no cambió, o si tanto el anterior como el nuevo superan $c_u^{[k]}$, la ruta estaba y sigue estando fuera de las k mejores y la clave no cambia; en otro caso se reconstruye, con un ordenamiento $\mathcal{O}(m \log m)$ sobre un búfer reutilizado para evitar reservas de memoria. Para $k = 3$ y $k = 4$ el salto se aplica siempre que la columna modificada no esté entre las k mejores rutas del cliente. Para $k = m$ ninguna ruta factible queda fuera, de modo que el agregador se reconstruye siempre que cambia el mejor costo de la columna: el arrepentimiento sobre todas las rutas es el operador de reparación más costoso.

Diferencias entre los arrepentimientos. El arrepentimiento 2 se mide sobre posiciones y los demás sobre rutas. El primero es sensible a la diferencia entre las dos mejores posiciones aunque estén en la misma ruta, y por tanto prioriza a los clientes cuya colocación *exacta* importa; los segundos miden cuántas rutas alternativas tiene realmente el cliente y priorizan a los que dependen de pocas rutas. Entre estos, k regula el horizonte de anticipación: con $k = 3$ o $k = 4$ solo cuentan las mejores alternativas, mientras que con $k = m$ el término $10^4(m - h)$ suele dominar la clave y el operador inserta primero, en esencia, a los clientes que caben en *menos* rutas, usando la suma de diferencias para desempatar. Frente a la inserción voraz, todos anticipan qué clientes quedarán sin alternativas, lo que es especialmente valioso con ventanas de tiempo estrechas y, sobre todo, con flota fija: en la fase 1 un cliente que se queda sin ruta factible permanece en el banco (Sección 3.1).

7.5. Resumen de las optimizaciones

La Tabla 5 reúne las optimizaciones descritas en esta sección y en la anterior. Todas son *exactas*: cambian el costo computacional, no la decisión que toma el operador. En las reparaciones con ruido, la perturbación forma parte de la definición del operador (se sortea una vez por entrada calculada) y la caché no introduce ninguna aproximación adicional.

Tabla 5: Optimizaciones algorítmicas de los operadores. U : clientes por insertar; $\bar{L}$: longitud media de ruta; m : número de rutas.

Componente	Técnica	Costo directo	Costo optimizado
Factibilidad de inserción	Llegadas, esperas y holguras por ruta	$\mathcal{O}(\bar{L})$	$\mathcal{O}(1)$
Arcos imposibles	Matriz de alcanzabilidad precalculada	—	filtro $\mathcal{O}(1)$
Extracción de peores	Lista ordenada incremental; solo se actualizan dos vecinos	$\mathcal{O}(qn \log n)$	$\mathcal{O}(n \log n + qn)^\dagger$
Extracción de Shaw	Orden de afinidad precalculado por instancia	$\mathcal{O}(qn \log n)$	$\mathcal{O}(qn)^\ddagger$
Voraz y arrepentimientos	Caché por (cliente, ruta); solo se recalcula la ruta modificada	$\mathcal{O}(U^2n)$	$\mathcal{O}(Un + U^2\bar{L})$
Agregadores	Actualización perezosa con prueba $\mathcal{O}(1)$	$\mathcal{O}(m)$ por cliente	$\mathcal{O}(1)$ habitual [§]
Borrado de pendientes	Intercambio con el último elemento	$\mathcal{O}(U)$	$\mathcal{O}(1)$

[†] El término qn corresponde a desplazamientos de memoria contigua, con constante muy pequeña.

[‡] Cota de peor caso del recorrido; en la práctica es mucho menor por el sesgo hacia rangos bajos. Se añade un precálculo único de $\mathcal{O}(n^2 \log n)$.

[§] Salvo en el arrepentimiento sobre todas las rutas, que reconstruye su agregador en cada cambio.

8. Síntesis: características que determinan el comportamiento

Esta sección recoge, a partir del análisis anterior, las características de diseño que condicionan los resultados de ambos métodos. Se distinguen las que son comunes, y por tanto explican el nivel de calidad que alcanzan los dos, de las que los diferencian.

8.1. Características comunes

1. **Rutas siempre factibles.** Las rutas nunca violan capacidad ni ventanas de tiempo. La única infactibilidad que admite la búsqueda son los clientes sin asignar, que tienen un peso fijo de 10^6 en f ; no hay otras penalizaciones que calibrar.
2. **Objetivo jerárquico escalarizado.** Los pesos de 10^6 por cliente sin asignar y de 10^4 por vehículo hacen que colocar un cliente domine sobre cualquier reducción de flota, y esta sobre cualquier mejora de distancia, en la aceptación y en la detección de mejoras.
3. **Reducción de flota en una fase propia.** La reparación no abre rutas, de modo que la flota es fija en cada etapa. La fase 1 elimina rutas una a una (retira la más corta y busca recolocar sus clientes con el ALNS completo) y se detiene al alcanzar la cota inferior de capacidad, al estancarse con 5 o más clientes pendientes durante 2000 iteraciones o al agotar $T_{\text{máx}}$ iteraciones. La fase 2 optimiza la distancia durante otras $T_{\text{máx}}$ iteraciones con la flota resultante. El número de vehículos obtenido depende, por tanto, de la eficacia con que cada método selecciona operadores *durante la fase 1*.
4. **Vecindarios de tamaño acotado.** Se extrae entre el 10 % y el 40 % de los clientes, con topes de 30 y 60: a partir de 300 clientes el vecindario tiene el mismo tamaño absoluto, lo que mantiene estable el costo por iteración al crecer la instancia.
5. **Catálogo complementario.** Hay operadores de diversificación (extracción aleatoria), de intensificación (extracción de peores, inserción voraz), de estructura (Shaw, grupos, rutas), específicos de las ventanas de tiempo y con anticipación (arrepentimiento 2, 3, 4 y sobre todas las rutas). El ruido en los costos de inserción, activo en la mitad de las iteraciones, diversifica cualquiera de las reparaciones.
6. **Recocido con reinicios.** La temperatura proporcional a la distancia inicial, el enfriamiento geométrico y el retorno a la mejor solución con recalentamiento a $0.5 T_0$ tras 1500 iteraciones sin progreso evitan que la búsqueda se congele en los periodos de estancamiento. La temperatura vuelve a T_0 al comenzar cada etapa de la fase 1 y al comenzar la fase 2.
7. **Eficiencia por iteración.** La evaluación de inserciones en tiempo constante, las cachés por ruta y las estructuras incrementales de los operadores de destrucción reducen el costo de la reparación en un factor del orden del número de rutas. Al ser optimizaciones exactas, no modifican la trayectoria de búsqueda: lo que aportan es que un presupuesto de hasta 50 000 iteraciones por corrida sea alcanzable en las instancias de mayor tamaño.

8.2. Diferencias entre los dos métodos

Como ambos métodos comparten todo lo anterior y parten de la misma solución inicial con la misma semilla, cualquier diferencia de resultados se origina en los aspectos de la Tabla 6.

Tabla 6: Comparación de los mecanismos de selección y aprendizaje.

Aspecto	ALNS clásico	ALNS con Q-learning
Unidad de decisión	Destrucción y reparación por separado (6 + 5 pesos)	Par conjunto (30 acciones)
Contexto	Ninguno	Estado binario: si la iteración anterior mejoró
Política	Sorteo proporcional a los pesos	ε -voraz, $\varepsilon \rightarrow 0.01$
Señal de crédito	Categoría del resultado: 33, 13, 9, 0	Magnitud de la mejora relativa en su propio criterio, logarítmica, ponderada global/local y creciente en el tiempo
Aceptación sin mejora	Se premia (9)	Se penaliza igual que un rechazo
Iteración sin éxito	Puntaje 0	Recompensa negativa $-\tau \Omega$: la mejora media reciente del mejor par alternativo
Frecuencia de actualización	Cada 100 iteraciones	Cada iteración
Memoria	Media móvil, $\lambda = 0.9$ por segmento	Media móvil, $\alpha = 0.5$ por visita
Horizonte	Inmediato	Descontado, $\gamma = 0.7$
Exploración	Permanente y proporcional ($w_{\min} = 0.01$)	200 iteraciones uniformes al inicio de cada fase y después residual
Cambio de fase	Sin reacción: los pesos continúan	Olvida las mejoras recientes y reinicia la exploración; conserva la tabla Q
Reparto de uso	Gradual entre todos los operadores	Concentrado en el par mejor valorado de cada estado

De la tabla se desprenden cuatro diferencias de comportamiento que conviene tener presentes al interpretar los resultados experimentales:

- **Sinergias entre operadores.** Solo el Q-learning puede aprender que un operador de destrucción rinde con un operador de reparación concreto: cada una de las seis destrucciones se valora por separado con cada una de las cinco reparaciones.
- **Qué se entiende por éxito.** El método clásico premia la aceptación, de modo que a temperatura alta refuerza a los operadores que generan soluciones aceptables aunque no mejoren. El Q-learning solo premia la mejora estricta y pondera su magnitud relativa, y penaliza cada fallo en proporción a lo que están rindiendo los demás pares.
- **Velocidad y concentración de la adaptación.** El método clásico modifica su distribución lentamente y mantiene en uso todo el catálogo; el Q-learning reacciona en pocas visitas y explota casi en exclusiva el par preferido de cada estado, cambiándolo cuando las penalizaciones lo desplazan. El primero es más estable y diverso; el segundo, más reactivo y más intensivo.
- **Adaptación al cambio de objetivo.** Al pasar de eliminar rutas a reducir distancia, el método clásico arrastra los pesos de la fase 1 y los corrige al ritmo de su media móvil; el Q-learning vuelve a explorar todos los pares de forma explícita antes de explotar. La fase 1 es, además, la que decide el número de vehículos, y en ella ambos mecanismos disponen de un número de iteraciones variable (a lo sumo $T_{\max}$) para aprender.

Estas afirmaciones describen el mecanismo, no su efecto medido. Para vincularlas con los resultados se recomienda acompañar las tablas comparativas con la frecuencia de uso de cada operador (o par) a lo largo de la búsqueda en ambos métodos, separada por fases, y con la duración de la fase 1.

Referencias

- [1] H. Gehring y J. Homberger. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. En *Proceedings of EUROGEN99*, pp. 57–64, 1999.
- [2] S. Kirkpatrick, C. D. Gelatt y M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [3] D. Pisinger y S. Ropke. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8):2403–2435, 2007.
- [4] S. Ropke y D. Pisinger. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472, 2006.
- [5] M. W. P. Savelsbergh. The vehicle routing problem with time windows: Minimizing route duration. *ORSA Journal on Computing*, 4(2):146–154, 1992.
- [6] P. Shaw. Using constraint programming and local search methods to solve vehicle routing problems. En *Principles and Practice of Constraint Programming – CP98*, LNCS 1520, pp. 417–431. Springer, 1998.
- [7] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.
- [8] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2.^a edición, 2018.
- [9] C. J. C. H. Watkins y P. Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.